

1 计算机视觉中的图像特征

1.1 填写函数的方式和原因

定义函数

函数定义由三个部分组成。

1. 函数声明以关键字 `function` 开头，定义调用函数的语法。
2. 函数体包含使用输入计算函数输出的命令。输出必须在函数体中定义。
3. `end` 关键字是函数的最后一行。

```
1 function out = myFunction(input)
2 % code that calculates out
3 end
```

调用优先级

在 MATLAB 中，设置了规则以解释任何命名项。这些规则称为函数优先顺序。可以使用以下顺序解决最常见的引用冲突：

变量

当前脚本中定义的函数

当前文件夹中的文件

MATLAB 搜索路径上的文件

1.2 计算机视觉入门之旅

计算机视觉入门之旅

课程简介：通过将典型 workflow（检测跟踪）应用于海龟爬向大海的视频，学习计算机视觉的基础知识。您将了解特征在计算机视觉中的作用、如何标记数据、训练目标检测器以及在视频中跟踪野生动物。

使用目标检测跟踪海龟

在本课程中，您将学习如何使用目标检测来跟踪视频中的目标。课程将从导入视频数据开始，逐步讲解此检测跟踪 workflow。

Import Video Data $\Rightarrow$ Label Ground Truth Data $\Rightarrow$ Train Detector $\Rightarrow$ Detect Objects in Each Frame $\Rightarrow$ Track Objects

1.2.1 创建视频读取器

可以使用 `VideoReader` 函数创建一个视频读取器，用于从文件中读取视频数据。

```
1 v = VideoReader("myVid.mp4")
```

创建视频读取器后，您可以使用 readFrame 函数从视频文件中提取帧。

```
1 v = VideoReader(filename)
2 fr = readFrame(v);
```

readFrame 的输出是图像，它是由像素值组成的数组。您可以在 readFrame 函数结尾使用分号，以避免 MATLAB 显示数百万个数字。

视频的一个帧就是一个图像。从视频文件中提取帧后，您可以像处理任何图像一样处理该帧。

```
1 imshow(frame)
2 %显示视频某一帧
```

在输出窗格的顶部，您可以看到 turtleVideo 的一些属性。例如，FrameRate 属性指示此视频的帧速率，即每秒显示的帧数

您可以用圆点表示法提取这些属性。

```
1 prop = obj.PropertyName
```

在上一个任务中，t 是 0.0333，即 $\frac{1}{\text{frame rate}}$ 秒。在第 1 行上创建视频读取器时，CurrentTime 属性的值为 0。

您可以使用 imshowpair 函数叠加两个图像，并以绿色和品红色显示它们的差异。

```
1 imshowpair(im1,im2)
```

读取特定帧

您可以反复调用 readFrame 以读取此视频的任一帧。但如何直接跳到第 90 帧呢？或如何回退到某个更早的帧？在本练习中，您将学习如何读取特定帧以及如何导航到视频中的任一帧。

在输出窗格的顶部，您可以在 turtleVideo 的输出中看到视频的总帧数和持续时间。变量 tMid 大约在视频的一半位置。

```
1 turtleVideo =
2   VideoReader - 属性:
3
4   常规属性:
5       Name: 'turtles.avi'
6       Path: '/users/mwa0000035781096/.training/.work'
7       Duration: 3.3333
```

```

8      CurrentTime: 0
9      NumFrames: 100
10
11 视频属性:
12      Width: 400
13      Height: 600
14      FrameRate: 30
15      BitsPerPixel: 24
16      VideoFormat: 'RGB24'

```

1.3 从参考图像中检测并提取特征

识别海龟的特征

要进行目标检测，您需要具有待检测目标的参考图像。参考图像应仅包含对检测该目标有用的特征。在此任务中，您将通过在视频帧中裁剪海龟周围的区域，来创建一个海龟的参考图像。

要指定要裁剪的矩形，需提供左上角的坐标以及矩形的宽度和高度（以像素为单位）。对于最小的 x 值和 y 值，您可以分别使用图像的列数和行数。

您可以通过 `imcrop` 函数指定要保留的矩形来裁剪图像。`imCropped = imcrop(im,rect);` 将矩形指定为一个四元素向量 `[xmin ymin width height]`。

```

1      rect = [xmin ymin width height];
2      imCropped = imcrop(im,rect);

```

您可以从矩阵或灰度图像中提取特征。由于 `turtle` 和 `frame` 是彩色图像，提取特征之前需将其转换为灰度图像。

您可以使用 `im2gray` 函数基于彩色图像创建灰度图像。

```

1      imGS = im2gray(im);

```

识别特征过程分为两部分：检测和提取。要检测图像中的类连通域特征，您可以使用 `detectSIFTFeatures` 函数。

输出是一系列感兴趣点的列表，其中可能包含尺度不变特征变换 (SIFT) 特征。

```

1      pts = detectSIFTFeatures(im)

```

现在您有了感兴趣点，可以使用它们来提取特征。感兴趣点标识了可查找特征的位置；一个点附近可能有多个特征，也可能一个特征也没有。

您可以对任何类型的特征使用 `extractFeatures` 函数。

第一个输出是特征矩阵。第二个输出是另一个 SIFTpoints 数组，其中包含这些特征的具体位置。

```
1 [feat,ftPts] = extractFeatures(im,pts)
```

您可以通过绘图对提取的特征可视化。大多数特征对应于海龟图像的某个区域。SIFT 特征是一种连通域特征。其他特征类型对应使用相同语法的函数。使用的特征类型取决于图像中的特征以及使用方法。

```
1 imshow(turtle)
2 hold on
3 plot(featurePoints)
4 hold off
```

您已从参考图像和原始视频帧中提取了特征，现在可以使用 matchFeatures 函数来匹配这些特征。

输出是一个包含匹配特征的索引的两列矩阵。indexedPairs 的第一列包含 feat1 的索引。第二列包含 feat2 的索引

```
1 indexedPairs = matchFeatures(feat1,feat2)
```

1.4 创建训练图像

脚本中的 dir 命令返回当前文件夹中.png 文件的列表。您可以在输出窗格中看到此时没有这样的文件。

您可以使用 objectDetectorTrainingData 函数从视频文件创建训练图像。此函数将指定的每一帧写入一个图像文件，忽略不包含检测目标的帧。此语法使用 SamplingFactor 名称-值参量，每 n 帧提取一帧。

第一个输出是创建的图像的列表。第二个输出包含这些图像的标签。

```
1 [ims,labs] = objectDetectorTrainingData( ...
2     gTruth,SamplingFactor=n);
```

您可以通过在 objectDetectorTrainingData 函数中使用额外的名称-值参量来自定义图像文件的组织方式。例如，您可以标准化文件名或将图像写入特定文件夹。

这里，创建的图像的名称以 basename 开头，并保存在 foldername 文件夹中。

```
1 [ims,labs] = objectDetectorTrainingData( ...
2     gTruth,SamplingFactor=n, ...
3     NamePrefix="basename", ...
4     WriteLocation="foldername");
```

训练检测器时，您需要将图像列表和框标签合并为一个变量。您可以使用 `combine` 函数合并图像和标签。

```
1 imsWithLabels = combine(ims, labs);
```

1.5 训练 ACF 检测器

聚合通道特征 (ACF) 目标检测器使用图像的颜色通道和其他二维表示的特征来检测目标。

您可以使用 `trainACFObjectDetector` 函数创建一个使用真实值图像训练的 ACF 检测器。

```
1 d = trainACFObjectDetector(imsWithLabels)
```

您可以使用 `detect` 函数将 `detector` 应用于图像。`[b,s] = detect(detector,im)` 第一个输出是一个由边界框组成的矩阵，边界框标识 `im` 中检测到的目标。第二个输出是一个由置信度分数组成的矩阵，置信度分数指示每次检测的强度。

检测器发现了海龟，但也误将一块岩石标注为海龟。

具有带注解的边界框的视频帧，这些边界框围绕四只海龟和一块岩石

默认情况下，目标检测训练分为四个阶段。训练阶段越多，准确度越好，但需要的运行时间也越长。

您可以在 `trainACFObjectDetector` 函数中使用 `NumStages` 名称-值参量来设置阶段数。

```
1 detector = trainACFObjectDetector( ...  
2     imsWithLabels, ...  
3     NumStages=n)
```

1.6 对检测目标进行后处理

图像中每个带注解的检测目标都标注有其分数。分数越高，表示检测到的目标是海龟的置信度越高。

您可以通过设置分数阈值来删除低置信度的检测目标。用直方图可视化分数有助于确定阈值。

```
histogram(score,10)
```

根据直方图，您可以决定保留分数大于 50 的检测目标。但将阈值设置得太高也不合适，这会增加检测器对新帧的限制。我们先尝试保留分数大于 30 的检测目标。

1.7 对检测目标计数

现在，您可以将数值变量和用双引号引起来的文本串联起来创建一个字符串，以表示检测到的海龟数量。

```
n = 50;
```

```
s = "There are" + n + " states in the USA.";
```

您将在下一个任务中使用此字符串来注解图像。

您可以使用 `insertText` 函数通过文本来注解图像。

```
imNew = insertText(im,[xmin ymin],textStr);
```

第二个输入是一个由文本框左上角的 `x` 坐标和 `y` 坐标组成的向量。第三个输入是您要包含的字符串。

1.8 什么时候读取完所有帧？

在下一个练习中，您将学习如何处理视频的每帧。您如何知道何时停下来？例如，输出窗格显示当前时间是 0.9333。在本练习中，您将确定是否已读取所有帧。

您尝试读取下一帧之前，可以使用 `hasFrame` 函数确定是否还有任何剩余帧。

```
yn = hasFrame(v)
```

输出是一个逻辑值，如果视频读取器中仍有待读取的帧，其值为 `true`；如果读取结束，则为 `false`。

1.9 对每帧应用检测算法

在本练习中，您将海龟检测算法应用于视频第一秒内的每一帧。

在工作区中，`detector` 变量已基于更多数据进行了训练。此检测器比您之前创建的检测器更准确，但花费的训练时间更长。

您可以使用 `while` 循环重复代码，直到满足某个条件。

此处，`condition` 为 `true` 或 `false`，只要为 `true`，循环内的代码就会重复。在此任务中，您需要重复执行代码，直到已遍历所有帧（可通过 `hasFrame` 函数确定是否已遍历所有帧）。

```
1 while condition
2     % code that does stuff
3 end
```

1.10 初始化卡尔曼滤波

在下一课中，您将使用卡尔曼滤波器跟踪此视频中的海龟。您通过跟踪能够知道海龟的位置，虽然它在故障期间检测不到。

要设置卡尔曼滤波器，您需要对目标的运动做一些假设。您觉得它是均匀加速运动（就像下落的物体）还是匀速运动？视频中的海龟在剪辑期间似乎以相同的速度逐帧移动。

您使用的运动模型会影响需要指定的关于运动和测量噪声的其他参数。您在前五个任务中定义参数，然后在任务 6 中使用它们初始化卡尔曼滤波器。

初始化卡尔曼滤波器所需的第二个假设是初始位置，它是根据边界框计算的。

`initialLoc = centroid`

课程将测量噪声定义为方差。质心出现了约 10 个像素的位移。因此，您可以对测量噪声参数使用

$10^2 = 100$

要初始化用于跟踪的卡尔曼滤波器，请使用 `configureKalmanFilter` 来指定您将跟踪的运动类型的假设。

```
1 kalmanFilter = configureKalmanFilter(MotionModel,...
2     InitialLocation,...
3     InitialEstimateError,...
4     MotionNoise,...
5     MeasurementNoise)
```